

Name: RIDIP BAISHYA

Roll No: CSB24038

Course: B.Tech in Computer Science and Engineering

Course Enrollment Dashboard in ReactJS

1. Introduction

This assignment demonstrates how to build a Course Enrollment Dashboard using ReactJS. The dashboard manages student information and their enrolled courses. React state management is used to store students and perform operations such as adding students, removing students, sorting by GPA, displaying unique courses, and filtering students based on course enrollment.

2. Student Data Structure

Each student is represented using the following structure:

```
{ id: number, name: string, enrolledCourses: Set<string>, gpa: number }
```

3. ReactJS Implementation

```
import React, { useState } from "react";

function CourseDashboard() {

  const [students, setStudents] = useState(new Map());

  const addStudent = (student) => {
    const newStudents = new Map(students);
    newStudents.set(student.id, student);
    setStudents(newStudents);
  };

  const removeStudent = (id) => {
    const newStudents = new Map(students);
    newStudents.delete(id);
```

```

    setStudents(newStudents);
  };

  const sortedStudents = [...students.values()].sort((a, b) => b.gpa - a.gpa);

  const uniqueCourses = [
    ...new Set(
      [...students.values()].reduce((acc, s) => {
        return [...acc, ...s.enrolledCourses];
      }, [])
    )
  ];

  const filterByCourse = (course) => {
    return [...students.values()].filter((s) =>
      [...s.enrolledCourses].includes(course)
    );
  };

  return (
    <div>
      <h2>Course Enrollment Dashboard</h2>

      <h3>Students Sorted by GPA</h3>
      {sortedStudents.map((s) => (
        <div key={s.id}>
          {s.name} - GPA: {s.gpa} - Courses: {[...s.enrolledCourses].join(", ")}
        </div>
      ))}

      <h3>Unique Courses</h3>
      {uniqueCourses.map((c, index) => (
        <div key={index}>{c}</div>
      ))}
    </div>
  );
}

export default CourseDashboard;

```

4. Explanation

The application stores students in React state using the `useState` hook. A `Map` is used internally to map student IDs to student objects. Students can be added and removed without mutating the original state by creating a new `Map` using the spread technique. Sorting is performed to display students based on GPA in descending order. The unique courses are computed using `Set` to avoid duplicates. The filter operation allows displaying students enrolled in a specific course.

5. Time Complexity Analysis

Filtering students by course requires checking each student in the list. If there are n students and each student has up to c courses, the filter operation checks the course membership for each student. Therefore the time complexity is $O(n \times c)$.

6. Conclusion

This assignment demonstrates the use of ReactJS state management and JavaScript data structures such as `Map` and `Set`. Functional programming methods like `map`, `filter`, and `reduce` help process data efficiently while keeping the code clean and easy to understand.